TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

BE Programme | Year III / Part II

EMBEDDED SYSTEM (CT 655)

QUESTION BANK WITH ANSWERS

Chapter 3: Software Design Issues

COURSE SYLLABUS AT A GLANCE

Unit	Topic	Hours	Marks
1	Introduction to Embedded System	3	4
2	Hardware Design Issues	4	8
3	Software Design Issues	6	8
4	Memory	5	8
5	Interfacing	6	8
6	Real-Time Operating System (RTOS)	8	12
7	Control System	3	8
8	IC Technology	3	8
9	Microcontrollers in Embedded Systems	3	8
10	VHDL	4	8
	Total	**45**	**80**

Chapter 3 Topics (6 Hours | 8 Marks):

- 3.1 Basic Architecture of General-Purpose Processor
- 3.2 Operation — Fetch, Decode, Execute Cycle
- 3.3 Programmer's View: Registers, ISA, Memory Map, Addressing Modes
- 3.4 Development Environment: Cross-Compiler, Linker, Debugger, Simulator, Emulator
- 3.5 Application-Specific Instruction-Set Processors (ASIP) — Types and Compromise
- 3.6 Selecting a Microprocessor — Criteria
- 3.7 General-Purpose Processor Design — Pipeline, DSP Features

Chapter 3: Software Design Issues

Q1. Explain the software development process with the view of embedded systems. Discuss briefly about programmer's view for general purpose processor.	**2082 Kartik (Back) | 2074 Bhadra (Regular) [4+4]**

ANSWER

Software Development Process for Embedded Systems:

1. Requirements Specification: Define functional requirements (inputs/outputs/behavior) and non-functional requirements (timing, power, cost, safety).
2. Architecture Design: Select processor, partition into tasks/ISRs, decide on OS/RTOS, define hardware-software interface.
3. Detailed Design: Design each module — algorithms, state machines, data structures, interrupt strategy.
4. Coding: Write in C/Assembly using a cross-development toolchain on the host PC.
5. Cross-Compilation: Cross-compiler (arm-none-eabi-gcc) compiles source to target object files (.o).
6. Linking and Locating: Linker combines .o files; locator assigns absolute ROM/RAM addresses per linker script.
7. Simulation/Debugging: Simulate on host; debug on hardware with JTAG debugger.
8. Flashing: Program final binary into target Flash memory using hardware programmer.
9. Testing and Verification: Functional tests, timing tests, boundary conditions, integration tests.

Programmer's View for General-Purpose Processor:

- Register Set: General-purpose registers (R0–Rn), PC, SP, Status/Flags register.
- Instruction Set Architecture (ISA): All valid instructions — arithmetic, logic, data transfer, branch.
- Memory Map: ROM (code), RAM (data/stack), memory-mapped peripheral registers.
- Addressing Modes: Immediate, Register, Direct, Register-Indirect, Base+Offset, Auto-increment.
- Flags Register: Carry (C), Zero (Z), Negative (N), Overflow (V) — tested by conditional branches.
- Calling Conventions (ABI): Parameter passing, return values, caller/callee-saved registers.
- Interrupt Model: Vectors, priority, ISR entry/exit, context save/restore.

Q2. Draw the basic architecture of general purpose processor. Explain the sub-operations of control unit with necessary illustration.

2081 Chaitra (Regular) | 2078 Baishakh (Back) [2+6]

ANSWER

Basic Architecture:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                  GENERAL-PURPOSE PROCESSOR                               │
│  ┌──────────────────────────────┐  ┌──────────────────────────────┐      │
│  │          DATAPATH            │  │      CONTROL UNIT (CU)       │      │
│  │  Register File (R0..Rn)      │  │  Instruction Decoder         │      │
│  │  ALU (+,-,AND,OR,SHL..)      │  │  Sequencer / FSM             │      │
│  │  PC | IR | MAR | MDR | SP    │  │  Micro-operation Logic       │      │
│  └──────────────┬───────────────┘  └──────────────┬───────────────┘      │
│                 │                                 │                      │
│                 Data/Address Bus                  Control Signals        │
│  ┌──────────────────────────────────────────────────────────────┐       │
│  │                      SYSTEM BUS                              │       │
│  │  Address Bus (unidirectional) | Data Bus | Control Bus       │       │
│  └──────────────────────────┬───────────────────────────────────┘       │
│                             │                                            │
│                        ┌────┴────┐                                       │
│                        │ MEMORY  │                                       │
└──────────────────────────────────────────────────────────────────────────┘
```

Sub-Operations of the Control Unit — Fetch-Decode-Execute Cycle:

1. Instruction Fetch (IF): MAR←PC; MDR←Memory[MAR]; IR←MDR; PC←PC+size

2. Instruction Decode (ID): CU reads opcode from IR; identifies instruction type; generates register file read addresses.

3. Operand Fetch: Source registers Rs1, Rs2 read from register file; effective address computed for memory ops.

4. Execute (EX): ALU control signals generated; ALU operates on operands; result and flags updated.

5. Memory Access (MEM): LOAD: MAR←EA; MDR←Memory[MAR]. STORE: MAR←EA; Memory[MAR]←Rs.

6. Write-Back (WB): ALU result (or loaded data) written to destination register Rd.

7. Branch: If condition TRUE: PC←target address. Else: continue sequentially.

Q3. Describe programmer's view in the context of software development for general purpose processor. Define Debugger, Simulator and Emulator.

2080 Chaitra (Regular) [5+3]

ANSWER

Programmer's View: (See Q1 for complete list.)

Debugger:

A software tool enabling controlled execution to find and fix bugs. Features: breakpoints, single-stepping, register/memory inspection, call stack trace, watchpoints. Examples: GDB, Keil µVision Debugger.

Simulator:

Host-PC software that models target processor behavior without real hardware. Executes target binary by interpreting instructions. Useful for early software testing. Limitation: may not be cycle-accurate.

Emulator (In-Circuit Emulator — ICE):

Hardware/software combination replacing the target processor on the actual board. Provides cycle-accurate execution, hardware breakpoints, and full execution tracing. Modern equivalent: JTAG On-Chip Debug (OCD) interface built into most MCUs.

Comparison:

Property	Simulator	Emulator (ICE/JTAG)
Hardware required	No (runs on host PC)	Yes (target hardware + debug probe)
Cycle accuracy	Approximate	Exact
Timing-sensitive debug	Not possible	Possible
Hardware interaction	Simulated peripherals	Real peripherals
Cost	Free/low	Higher (JTAG probe)

Q4. How does ASIP serve as a compromise between general-purpose and single-purpose processors?	**2079 Ashwin (Back) | 2073 Bhadra (Back) [2+6]**

ANSWER

General-Purpose Processor	ASIP	Custom Single-Purpose Processor
Most flexible — runs any program	Customized for one application domain	Fixed — only one program
Highest NRE cost (complex), moderate unit	Moderate NRE and unit cost	Low NRE if reused; lowest unit cost
Lowest perf/power ratio	Balanced performance and power	Best performance, lowest power
Programmable in any language	Programmable with custom ISA tools	Not user-programmable
Example: ARM Cortex-A, x86	Example: DSP TI C6748, GPU	Example: GCD circuit, AES engine

ASIP achieves the compromise by:

- Adding custom instructions for the domain (e.g., MAC in one cycle for DSP).
- Removing unused GPP instructions to reduce area and power.
- Custom register file or ALU widths for the domain.
- Maintaining full programmability — unlike fixed hardware.

Different Forms of ASIP:

- Microcontroller: GPP core + peripherals on one chip. Examples: STM32, PIC, AVR.
- DSP: MAC unit, Harvard architecture, zero-overhead loops. Examples: TI TMS320, ADSP.
- Media Processor: SIMD extensions for video/audio. Example: ARM with NEON.
- Network Processor: CAM lookups, queue management, protocol parsing.
- GPU: Thousands of simple parallel cores for graphics and GPGPU.

Q5. Explain software development procedure showing implementation and verification phases. Describe the principle of pipeline.	**2081 Ashwin (Back) [5+3]**

ANSWER

Implementation Phase:

- Coding: Write C/Assembly source code per the detailed design.
- Cross-Compilation: Cross-compiler generates target object files (.o).
- Linking: Linker combines .o files, resolves symbol references → ELF file.
- Locating: Linker script assigns .text to ROM, .data/.bss to RAM, defines stack/heap.
- Binary Generation: objcopy converts ELF → .bin or .hex for programming.
- Flashing: Hardware programmer writes binary to target Flash.

Verification Phase:

- Simulation: Test logic on software simulator before hardware.
- Hardware Debugging: JTAG debugger — breakpoints, register/memory watch.
- Unit Testing: Test each function with known inputs and expected outputs.
- Integration Testing: Test inter-module interfaces and shared data.
- Timing Verification: Oscilloscope/logic analyzer confirms real-time deadlines.
- Code Coverage: All branches exercised during testing.

Principle of Pipelining:

```
Stage:  IF    ID    EX    MEM   WB
Inst1: [IF]  [ID]  [EX]  [ME]  [WB]
Inst2:       [IF]  [ID]  [EX]  [ME]  [WB]
Inst3:             [IF]  [ID]  [EX]  [ME]  [WB]

CPI(ideal)=1   Speedup ≈ N_stages
```

- IF: Fetch instruction; ID: Decode + read registers; EX: ALU; MEM: Load/store; WB: Write result.
- Hazards: Data (RAW) → forwarding/stall; Control (branch) → prediction/flush; Structural → duplicate hardware.

Q6. How can pipeline be implemented to improve performance? Write down the features of a DSP.	**2079 Chaitra (Regular) | 2078 Jestha (Back) [4+4]**

ANSWER

Pipeline Implementation:

- Divide execution into stages of equal latency — minimizes the critical path of the slowest stage.
- Insert pipeline registers between stages to hold intermediate data — enables parallel operation.
- Clock period = longest stage delay. Throughput → 1 instruction/cycle (ideal CPI = 1).
- Handle data hazards via forwarding paths; handle control hazards via branch prediction + pipeline flush.

Speedup = N_stages × (1 / (1 + stall_rate)) → maximum speedup = N_stages (ideal).

Features of a DSP:

- Harvard Architecture: Separate instruction and data memory buses — simultaneous instruction fetch and data access.
- MAC Unit (Multiply-Accumulate in 1 cycle): a = a + (b×c) in a single clock — essential for FIR/IIR filters, FFT.
- Zero-Overhead Hardware Loops: Dedicated loop counter — branch and decrement without consuming extra cycles.
- Circular Buffer Addressing: Hardware ring buffer management for convolution and streaming algorithms.
- Bit-Reversal Addressing: Hardware support for FFT butterfly index reversal.
- Multiple Memory Banks: Simultaneous access to multiple operands per cycle.
- Fixed-Point Saturation Arithmetic: Prevents signal wrap-around distortion on overflow.
- High-Speed Serial Interfaces: McBSP, I²S for real-time audio/signal acquisition.

Q7. Explain the architecture of general purpose processor. What are the criteria to select the processor?	2077 Chaitra (Regular) [4+4]

ANSWER

Architecture: (See Q2 for full block diagram and description.)

Criteria for Selecting a Processor:

- Performance: Clock speed, MIPS, and instruction mix must meet real-time deadlines.
- Power Consumption: Active power, sleep modes, and wake-up time must match the power budget.
- Cost: Unit cost and development cost (toolchain, eval boards) must fit project budget.
- On-Chip Peripherals: Required peripherals (ADC, UART, SPI, I2C, PWM, CAN) integrated to minimize BOM.
- On-Chip Memory: Sufficient Flash and SRAM to avoid external memory chips.
- Development Ecosystem: Mature compiler, debugger, RTOS, datasheets, application notes, community.
- Operating Environment: Temperature, voltage, EMI immunity, package type.
- Vendor Longevity: Long-term availability (10+ years) for products with extended service life.

Q8. Explain the basic operations carried by the controller of a general purpose processor. Describe different forms of ASIP.	2078 Poush (Back) [4+4]

ANSWER

Controller Operations: (See Q2 — all 7 sub-operations.)

Different Forms of ASIP: (See Q4 — all 5 forms.)

Q9. Explain the embedded software development toolchain with diagram. What is a cross-compiler and why is it needed?	Model Question | 2076 Ashwin (Back) [4+4]

ANSWER

Embedded Software Development Toolchain:

```
Source Code (.c/.h/.S)  ←  Written on HOST PC
        ↓
  [Cross-Compiler]   e.g., arm-none-eabi-gcc
        ↓
```

```
Object Files (.o)  — Target machine code
      ↓
[Linker + Locator]  — Linker script (.ld)
      ↓
Executable (ELF)   — Symbols + debug info
      ↓
[objcopy]  →  Binary (.bin) or Intel HEX (.hex)
      ↓
[Programmer/JTAG] → Flash/ROM on TARGET HARDWARE
      ↓
[Debugger (GDB + OpenOCD / J-Link)]  ← Debug on target
```

Cross-Compiler:

A cross-compiler is a compiler that runs on a host machine (e.g., x86 Linux/Windows PC) but generates executable code for a different (target) architecture (e.g., ARM Cortex-M, MIPS, RISC-V).

Why is it needed?

- The target embedded processor is often too resource-constrained (few KB RAM, simple OS) to run a full compiler itself.
- Development productivity: Fast host machine compiles embedded code much quicker than the target MCU could.
- Before hardware exists: Software can be compiled and tested (in simulator) before the physical hardware board is built.
- Multiple targets: One host cross-compiler can generate code for many different target architectures.

Example: arm-none-eabi-gcc runs on a Linux PC and generates binary code for ARM Cortex-M microcontrollers.

Q10. Differentiate between RISC and CISC instruction set architectures. Which is commonly used in embedded systems and why?	**Model Question | 2075 Poush (Back) [4+4]**

ANSWER

Property	RISC	CISC
Instruction set	Small, simple, fixed-length	Large, complex, variable-length
Execution	1 instruction / 1 clock (mostly)	Multi-clock instructions common
Memory access	Load-Store architecture only	Direct memory operands allowed
Registers	Many general-purpose registers	Fewer, more specialized registers
Pipeline	Easy to pipeline	Harder to pipeline (variable length)
Code density	Lower (more instructions needed)	Higher (complex single instructions)

Hardware complexity	Simpler hardware	Complex decoder/microcode
Examples	ARM, MIPS, RISC-V, AVR	x86, 8051, VAX

RISC in Embedded Systems:

RISC architectures (especially ARM Cortex-M and Cortex-A) dominate embedded systems because:

- Simple pipeline = lower power consumption and smaller silicon area.
- Predictable execution times (WCET analysis is easier) — essential for real-time systems.
- ARM Thumb/Thumb-2 ISA provides compact 16/32-bit mixed instructions for good code density.
- Vast ecosystem: ARM is used in over 200 billion chips — enormous toolchain and community support.

Q11. Explain interrupt handling in embedded systems. Differentiate between polling and interrupt-driven I/O.	**Model Question | 2077 Baisakh (Back) [4+4]**

ANSWER

Interrupt Handling in Embedded Systems:

An interrupt is a hardware signal that causes the processor to suspend its current execution and transfer control to an Interrupt Service Routine (ISR) to handle the event.

Interrupt Handling Sequence:

10. Peripheral or timer raises an interrupt request (IRQ) signal.
11. Processor completes current instruction, then checks interrupt flags.
12. If interrupt enabled (GIE set) and priority sufficient: CPU pushes PC and status register onto stack.
13. CPU fetches ISR address from the interrupt vector table.
14. ISR executes: reads the event, clears the interrupt flag, takes action.
15. ISR returns (RETI): CPU pops PC and status register from stack, resumes interrupted task.

Polling vs Interrupt-Driven I/O:

Property	Polling	Interrupt-Driven I/O
Mechanism	CPU continuously checks peripheral status register	Peripheral signals CPU when ready
CPU utilization	Wasted — busy waiting even if peripheral not ready	Efficient — CPU free until event
Latency	Variable (depends on polling rate)	Low and deterministic

Complexity	Simple to implement	More complex (ISR, vector table)
Power	Higher (CPU active during polling)	Lower (CPU can sleep)
Best used when	Peripheral always active, few peripherals	Infrequent events, power-sensitive
Examples	Simple button check in main loop	UART receive, ADC conversion done

Q12. Explain memory-mapped I/O and port-mapped (isolated) I/O with examples. Which is used in ARM processors?	**Model Question | 2074 Chaitra (Back) [4+4]**

ANSWER

Memory-Mapped I/O (MMIO):

Peripheral registers are placed in the same address space as memory. The CPU accesses peripherals using standard load/store instructions with specific addresses.

```
// ARM Cortex-M example: Toggle GPIO pin via MMIO
#define GPIOA_ODR  (*(volatile uint32_t *)0x40020014)
GPIOA_ODR |= (1 << 5);  // Set pin PA5 high
```

- Advantage: No special I/O instructions needed — any load/store works. Simpler CPU design.
- Advantage: Peripheral registers can be accessed with pointers — easy in C.
- Disadvantage: Reduces available memory address space.

Port-Mapped (Isolated) I/O:

Peripheral registers occupy a separate I/O address space, accessed using dedicated IN/OUT instructions.

```
// x86 assembly: Read from I/O port 0x60 (keyboard)
IN AL, 0x60   ; Read keyboard scan code into AL
OUT 0x61, AL  ; Write to port 0x61
```

- Advantage: I/O space is separate from memory — no address space reduction.
- Disadvantage: Requires special IN/OUT instructions — not available in all architectures.

ARM Processors use Memory-Mapped I/O exclusively.

All ARM Cortex-M processors (STM32, nRF52, LPC) use MMIO. The entire peripheral register bank is mapped in the address range 0x40000000–0x5FFFFFFF as defined by the ARM Cortex-M memory map. This makes embedded C programming clean and portable.

Summary: Topics Covered in Chapter 3

S.N.	Topic	Exam(s) Asked	Marks
1	Software Development Process / Phases	2082, 2081, 2080, 2074, 2073 (annual topic)	4–8
2	Programmer's View (ISA, registers, memory, modes)	2082, 2080, 2079, 2074	4–6
3	GPP Basic Architecture + Block Diagram	2081, 2078, 2077, 2072	2–4
4	Control Unit Sub-Operations (IF/ID/EX/MEM/WB)	2081, 2080, 2078	4–6
5	Debugger / Simulator / Emulator (comparison)	2080, 2070	3–5
6	ASIP — Compromise + Forms (MCU/DSP/GPU)	2079, 2078, 2073	2+6 / 4+4
7	Pipelining — Stages, Hazards, Performance	2081, 2079, 2078	3–5
8	DSP Features (8 key features)	2079, 2078	4
9	Processor Selection Criteria	2077, 2079, 2072	4
10	Embedded Toolchain + Cross-Compiler	2076, Model Q	4+4
11	RISC vs CISC	2075, Model Q	4+4
12	Interrupt Handling + Polling vs Interrupt I/O	2077, Model Q	4+4
13	Memory-Mapped vs Port-Mapped I/O	2074, Model Q	4+4